

Computer Science

Internal assessment

Criterion A

Client and scenario

The client, Yongsuk Kim, a friend and classmate of mine, is passionate about war-themed strategy games. In our first interview, he mentioned that he enjoys designing and creating original turn-based strategy game ideas on paper or via other physical mediums; however, he often struggles in being able to enjoy these games with his friends (see Appendix a.i.i). He mentioned that the key barriers are that physical games are less accessible, requiring physical materials as well as being more cumbersome to transport (Appendix a.i.ii). There are other disadvantages, such as when he highlighted, “Digital products also have saveability, in that I can easily save the state of a game when it’s not finished, whereas that option doesn’t exist as much for physical games” (Appendix a.i.iii).

He highlighted that a digital product, such as an application, would be capable of addressing many of these issues, such as being able to save games mid-round as digital files which can be read by the software, as well as being easy to access, download and play on portable devices such as a laptop via pass-and-play (see Appendix a.i.iii). The digital product would allow the rules of the game to be carried out automatically, allowing it to act as a far more functional product as well as a more replayable experience (Appendix a.i.i; a.i.iii). Therefore, the optimal way of addressing the client’s needs would be to recreate their strategy game concept into an application that can be easily run.

Rationale

The solution to the scenario is to develop a digital application in the form of a 2D strategy game that runs the rules and systems of the client’s design automatically.

To form the main basis for the game itself, I will be using pygame in my technology stack, which is a python library ideal for supporting 2D games and its corresponding visual and logical elements through modular programming (About - Wiki).

Pygame is also an easy library to use when it comes to supporting media assets necessary for the game such as sounds and sprites. Python is an object-oriented programming language, allowing me to easily take advantage of classes and methods in order to form the logic of the game including for each of the individual pieces and tiles as well as modularly implement the interactions between classes. This makes it preferable to languages like Javascript, which while

common in webgame development, does not offer a complete object-oriented programming framework (Raval). Libraries such as pyInstaller allow the main python files to be converted into a macOS executable file, allowing the client to run the game without needing to install the python interpreter.

A successful implementation of pure python + pygame to address a similar problem can be seen in Appendix b., an isometric-view strategy game with save functionality, different menus, dynamic piece interactions, and strategy-based gameplay with tiles. The work of Ben Ryder and Miklos Horvath on the game, “Conqueror of Empires”, demonstrate the versatility of this technology stack in addressing most if not all of the specifications for the context of developing a 2D strategy game (Ryder and Horvath).

Success criteria

1. The system must be able to compile and execute on the client’s platform, which is a 14-inch Macbook Pro laptop model from 2021, using the Apple M1 Pro chip.
2. The system must be carrying out all the game rules automatically, including actions such as movement of pieces, deployment of troops, combat system.
3. The system must be able to save the current game state if a game is not yet complete. The file should also be able to be reloaded back into the software and the game continued.
4. The system must have a general visual aesthetic style resembling that of WW2 table-top games including similar fonts and color palette.
5. The system must be presented as an easy to use single package or unit, such as a single zip file with a clear way of executing the program.
6. The system must have toggleable game settings, among which there should be a clear option to choose to play on 3 different boards or maps, with one map being smallest, one map being medium sized and one map being large sized.
7. The system must have gameplay offering a variety of terrain, each with slightly different properties including field tiles, urban tiles, mountain tiles, water tiles as well as modifiers such as reinforcement point tiles, roads and railways.
8. The system must have gameplay incorporating a variety of deployable units or pieces, which are purchased from the shop and have different properties, such as different movement speed, health points and range of attack.
9. The system must have sound effects accompanying actions performed by the player. Different unit types should have specific sound effects, such as a tank making tank-like noises, or infantry making foot-step noises.
10. The system must feature royalty-free music playing to accompany the atmosphere of the game.
11. The system must contain a title screen that is the first screen visible when the application is launched, which should also have buttons that allow the player to create a new game, load a saved game, and adjust settings.
12. The system must be able to support anywhere between 2 or 3 players in a 1v1 setting as well as a 1v1v1 setting with options for hostile as well as diplomatic interaction in-game.

Criterion B

Record of tasks

Task number	Planned action	Planned outcome	Time estimated	Target completion date	Criterion
1	Reach out to the Client and discuss needs.	Meet with the potential client, brief them on the project, briefly brainstorm possible candidate problems and solutions.	1 hr	18th March, 2024	A
2	Complete the initial IA proposal to the teacher.	Compile ideas gathered from brainstorming with the client, and get approval from the teacher in regards to the general outline, client, and technical choices.	1 hr	19th March, 2024	A
3	Conduct the first official interview with the client	Ask questions and learn more specifically in regard to the client's needs.	30 minutes	12th April, 2024	A
4	Conduct a second official interview with the client	Ask questions to gain insight into the design specifications that will be necessary for a satisfactory product.	1 hour	14th April, 2024	A
5	Summarise results from interviews, formulate	Concretely outline the client, scenario, rationale and success	1 hr 30 min	18th April 2024	A

	success criteria	criteria.			
6	Gather secondary research to justify success criteria and approach.	Included supporting research	1 hr 30 min	29th April 2024	A
7	Create a Wireframe for the UI	Outlined templates for the different UI layouts that will be incorporated into the product.	1 hr 30 min	30th April 2024	B
8	Complete annotation of the wireframe for the UI	Annotated templates for interactive features, etc.	1 hr	14th May 2024	B
9	Design Pseudocode for Interactive UI elements	Knowledge of how to implement necessary interactive UI elements in the product	1 hr	1st June 2024	B
10	Outline UML Class Diagrams and ER relationships	Have a plan of what classes must be defined along with specified attributes and methods, as well as their interactions with other classes.	2 hr	11th June 2024	B
11	Begin creating a flowchart for game methods and processes	Draft of the logic outlining functions executed from user inputs.	2 hr	18th June 2024	B
12	Complete flowchart for game methods and processes (and any other app processes)	Completed flowcharts that plan the logic that outlines moving pieces, battling pieces and other in-game logical	1 hr	5th July 2024	B

		functions.			
13	Design data storage and necessary algorithms in pseudocode.	A clear plan for storing/saving game data.	1 hr 30 min	10th July 2024	B
14	Outline Testing Plan to address Success Criteria	Methods for testing each feature that was outlined in the success criteria (at the end of criterion A)	1 hr	12th July 2024	B
15	Create new project, program Application class and Screen class.	An Application Class that is able to display an empty screen/window when main.py is run.	1 hr 30 mins	20th July 2024	C
16	Create graphics for all 6 (non-interactive) screens.	Basic visual skeleton for the application's operation	3 hr	21st July 2024	C
17	Program classes for all 6 screens.	6 classes that inherit from the screen class, that can then be displayed.	2 hr	23rd July 2024	C
18	Implement interactiveObject and Button class	Buttons that can transport the user between the different screens.	2 hr	26th July 2024	C
19	Implement Checkbox class and add to interactive elements on	Working checkboxes on the settings screen.	1 hr	29th July 2024	C
20	Create and display the game instructions on	Functioning 'how to play' screen accessible via the home page.	1 hr	31st July 2024	C

	the how to play screen.				
21	Create a 'select box' class and use to create load game screen.	A loadgame screen that can correctly display the files in data/gamestates that can be selected.	2 hr	3rd August 2024	C
22	Add up and down buttons to the load game screen.	Up and down buttons that allow more than 8 files to be displayed and selected.	30 minutes	4th August 2024	C
23	Create map selection options, and implement a text box class	Worked on the 'new game' screen: Created a selection option for the 3 different maps, implemented working textbox field for player names	3 hr	7th August 2024	C
24	Add team and color adjustment buttons to 'new game screen' and 'create game button'	All interactive features on the 'new game' screen are now completed, leaving only the game screen itself to program.	2 hr	9th August 2024	C
25	Create the (non-game) interface for the gameboard screen.	The gameboard has a 'save and exit' button and a 'help' button that redirect to those screens.	1.5 hr	15th August 2024	
26	Create a gameboard class with basic methods outlined in design	A gameboard class/object that exists as a component of the Game Screen.	1 hr	22nd August 2024	C

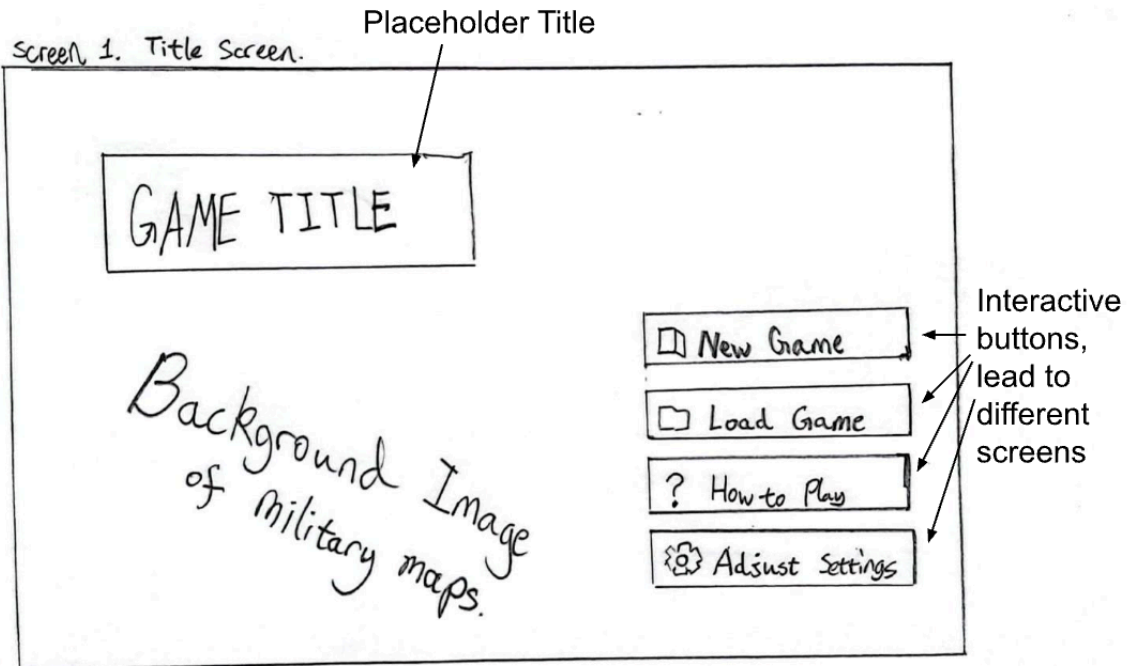
27	Generate the tile map, and have interactive tiles that can be selected and acted upon.	An interactive map/game board that can understand user commands.	5 hr	29th August 2024	C
28	Create a system for alternating between player turns, and create the basic troop class.	System that can support multiple-player gameplay, as well as generic troops.	4 hr	12th September 2024	C
29	Implement and load graphics for the troops into the game.	Graphically visible troops with the different gameplay attributes outlined by the client.	4 hr	25th September 2024	C
30	Program game currencies, as well as unit movement points and health points.	There is a game currency that can be used to spend on deploying troops, which has limited movement points and health points.	4 hr	29th September 2024	C
31	Create the UI for the shop, and program deployment of troops on empty tiles.	There is a fancy shop UI that can be pressed to allow the user to select a reinforcement tile for deployment.	4 hr	3rd October 2024	C
32	Program BFS or Dijkstra's algorithm for path finding between tiles.	It is possible to know the distance between 2 tiles, not just by spacial displacement, but by distance imposed by barriers and weighted costs	5 hr	10th October 2024	C

		between neighbouring tiles.			
33	Program inter-troop interactions as well as gameboard interactions	Troops can attack each other once per turn, and can also move to and capture different tiles.	5 hr	17th October 2024	C
34	Implement game-winning objectives	The game now knows to eliminate players if they have no captured tiles.	3 hr	24th October 2024	C
35	Work on serialization for all classes and game states.	Each class now has a json representation that can be used to save its state.	5 hr	31st October 2024	C
36	polish/bug fixing	All the key functionalities of the product are working as intended.	4 hr	7th November 2024	C
37	polish/bug fixing	Any extra polish, regarding graphics, sound effects and background music are added to give the feeling of a complete product.	3 hr	8th November 2024	C
38	Meet with client to try the product, attempt to send online.	Client now has access to the product, and may test it and try it in their own time.	1 hr	14th November 2024	E
39	Create notes for improvement in product feedback interview with client.	Useful, direct feedback on the pros and cons regarding functionality and aesthetic of the final product.	1 hr	21st November 2024	E

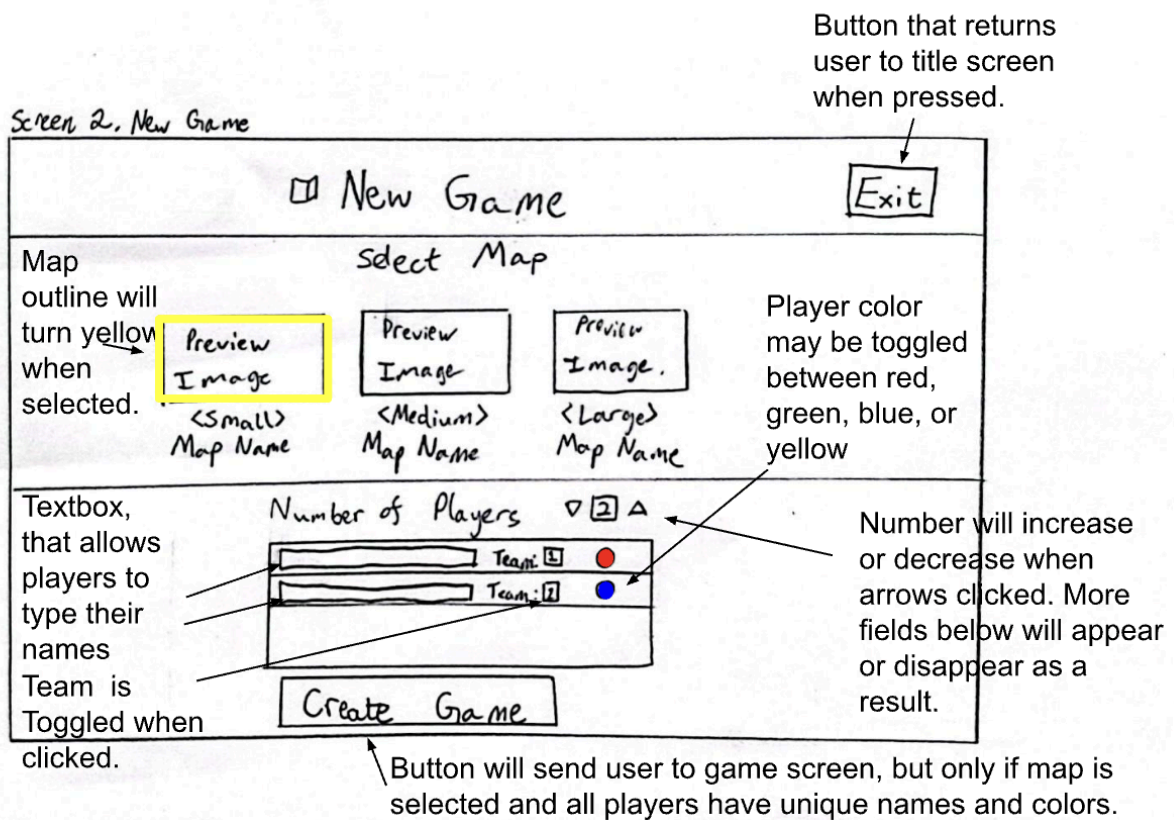
40	Screen record the product video.	A video showing proof of the completed software.	1 hr	22nd November 2024	D
----	----------------------------------	--	------	--------------------	---

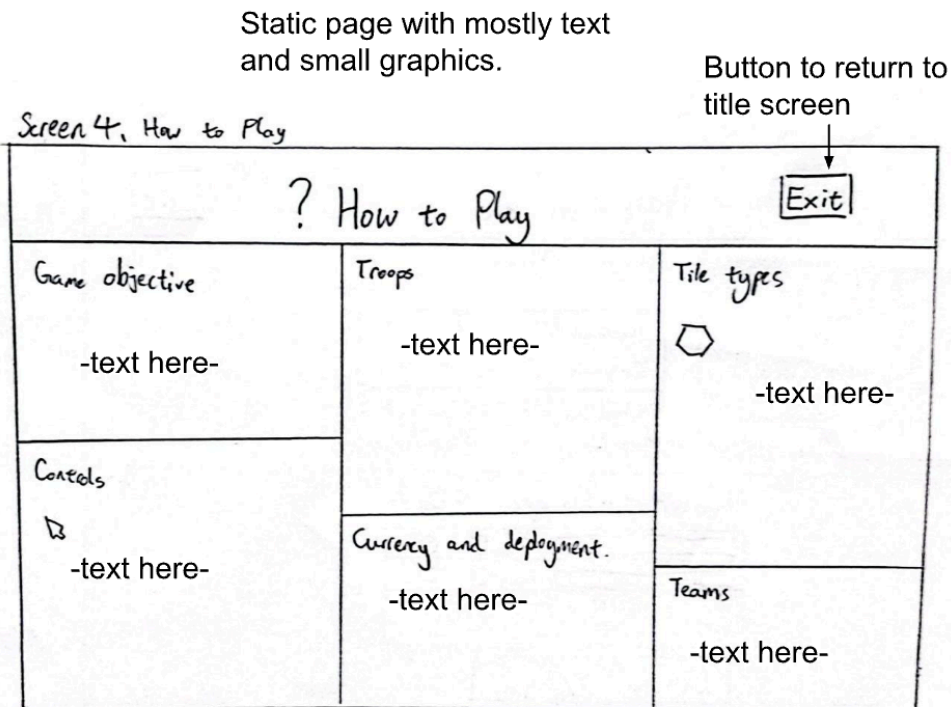
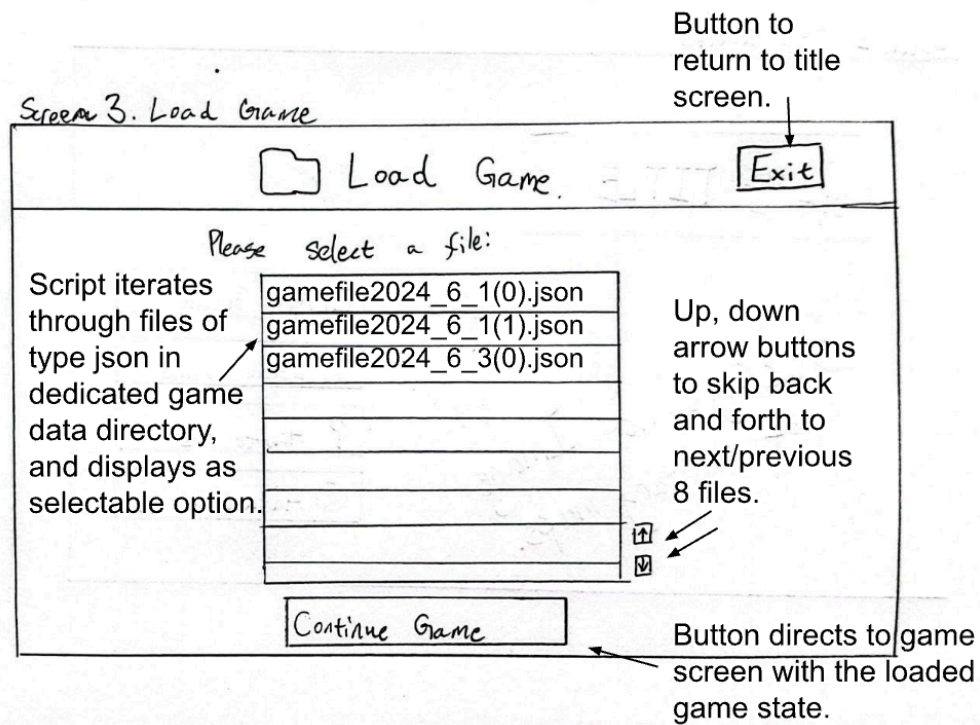
Design diagrams

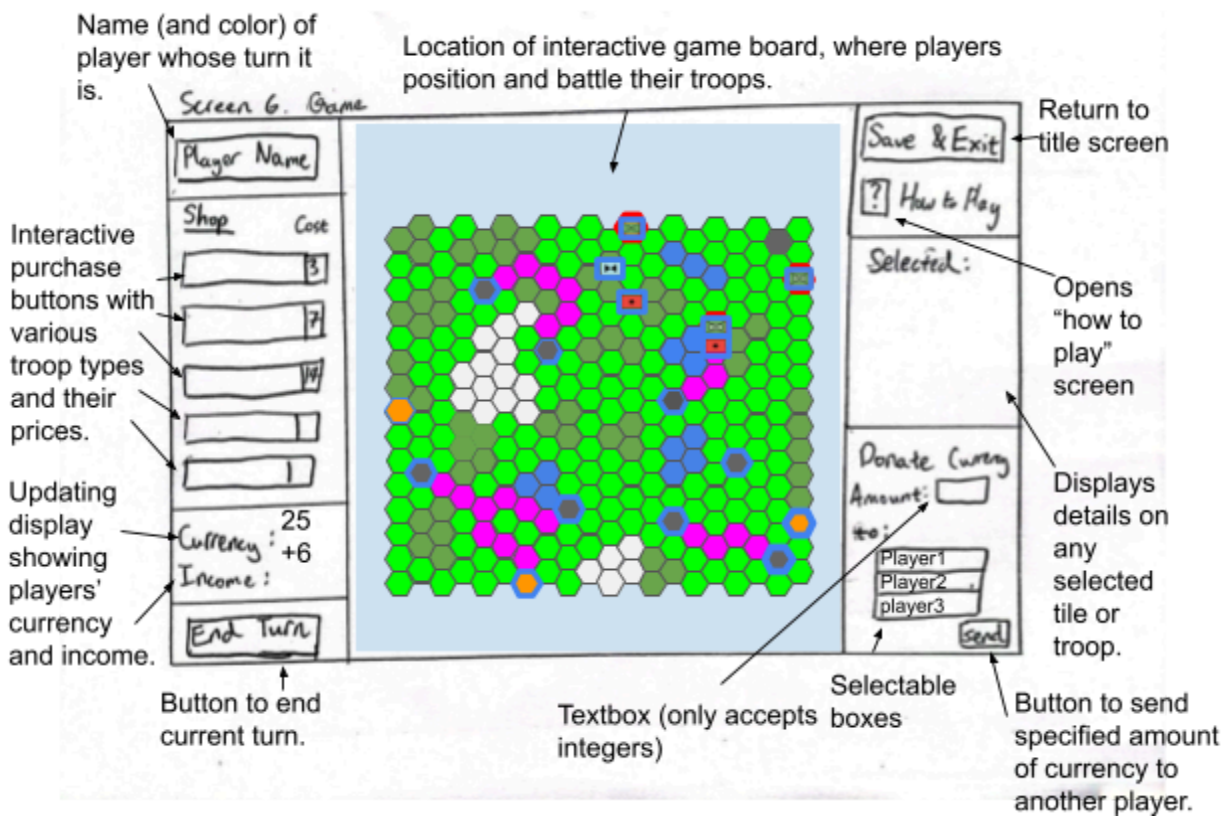
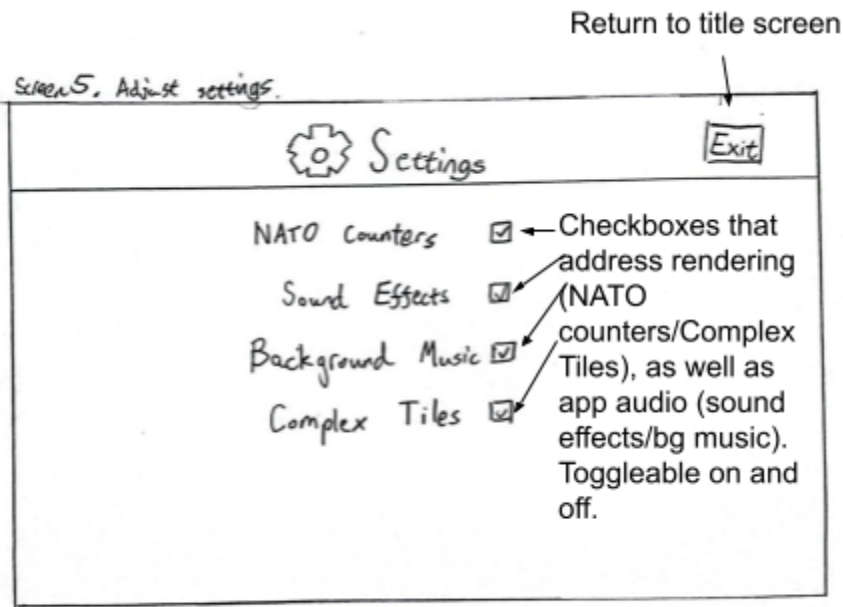
UI Wireframe Diagrams



Large Background Image across whole screen helps it feel similar to a board game box.







Interactive UI Event Handlers Pseudocode.

There are 3 main types of interactive components in the UI (other than the game board itself): buttons, text boxes and checkboxes.

Pseudocode for handling UI events.

```
interactive_elements = [button1, button2, textbox1, ...]

# running loop
running = true
while running:
    for event in event.get():
        for uiElement in interactive_elements:
            uiElement.handle_event(event) #pass the event to each object

    for uiElement in interactive_elements: #update each object in list
        uiElement.update()
        uiElement.render()
```

← List to store UI features (buttons, textboxes)

← Pass on each event to each object to handle separately.

← Update and render all objects

Broad class definition for interactive objects

```
class InteractiveObjectUI:
    def __init__(self, function, x, y, w, h):
        self.rect = pygame.Rect(x, y, w, h)
        self.color = COLOR_INACTIVE
        self.function = function
        self.active = False

    def handle_event(self, event):
        if event.type == pygame.MOUSEBUTTONDOWN:
            if self.rect.collidepoint(event.pos):
                self.active = not self.active

    def update(self):
        if self.active:
            self.color = COLOR_ACTIVE
            self.function() # executes its function
            self.active = False
        else:
            self.color = COLOR_INACTIVE

    def render(self, screen):
        pygame.draw.rect(screen, self.color, self.rect, 2)
```

Initialization includes defining a rectangle, a base color, a function, and a boolean active state

If the event is a mouse click that collides with self.rect, then toggle active state

If active, change color and execute the function once

UI elements inherit from the InteractiveObjectUI class. For example, for Button:

```

class Button(InteractiveObjectUI): # Class inheritance
    def __init__(self, function, x, y, w, h, graphic):
        super().__init__()

        # scale the graphic to button size
        pygame.transform.scale(graphic, (w, h))
        self.graphic = graphic

        self.pressed = False

# override inherited method (dynamic polymorphism)
def handle_event(self, event):
    if event.type == MOUSEBUTTONDOWN:
        if self.rect.collidepoint(event.pos):
            self.pressed = True
    elif event.type == MOUSEBUTTONUP:
        self.pressed = False
        self.active = True

def update(self):
    if self.pressed:
        self.color = COLOR_ACTIVE
    elif self.active:
        self.color = COLOR_INACTIVE
        self.function()
        self.active = False
    else:
        self.color = COLOR_INACTIVE
def render(self, screen):
    screen.blit(self.graphic, (x, y))
    pygame.draw.rect(screen, self.color, self.rect, 2)

```

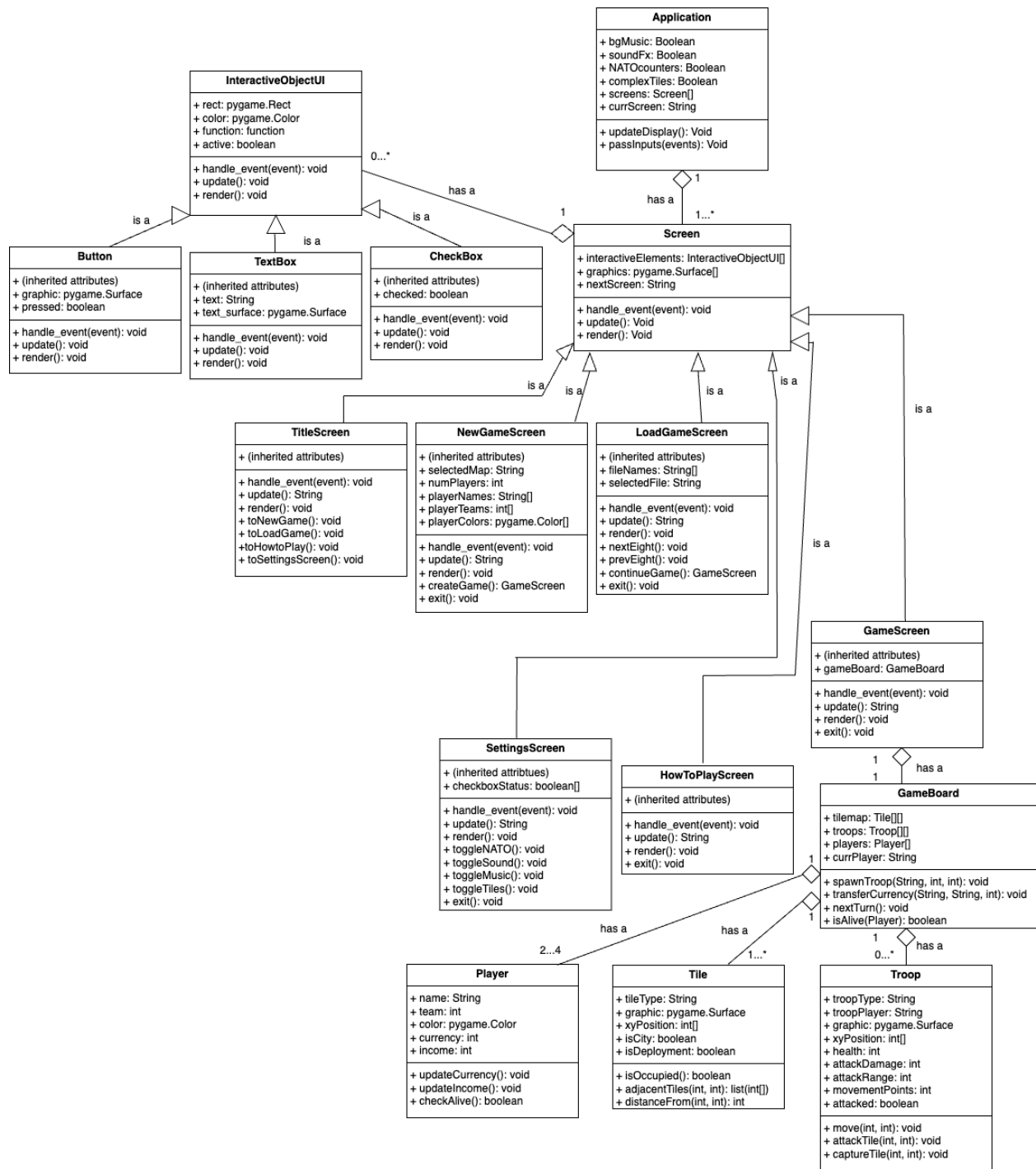
Inherit parent class methods

Graphic is a scaled image file

Button executes when the mouse is released (after being pressed)

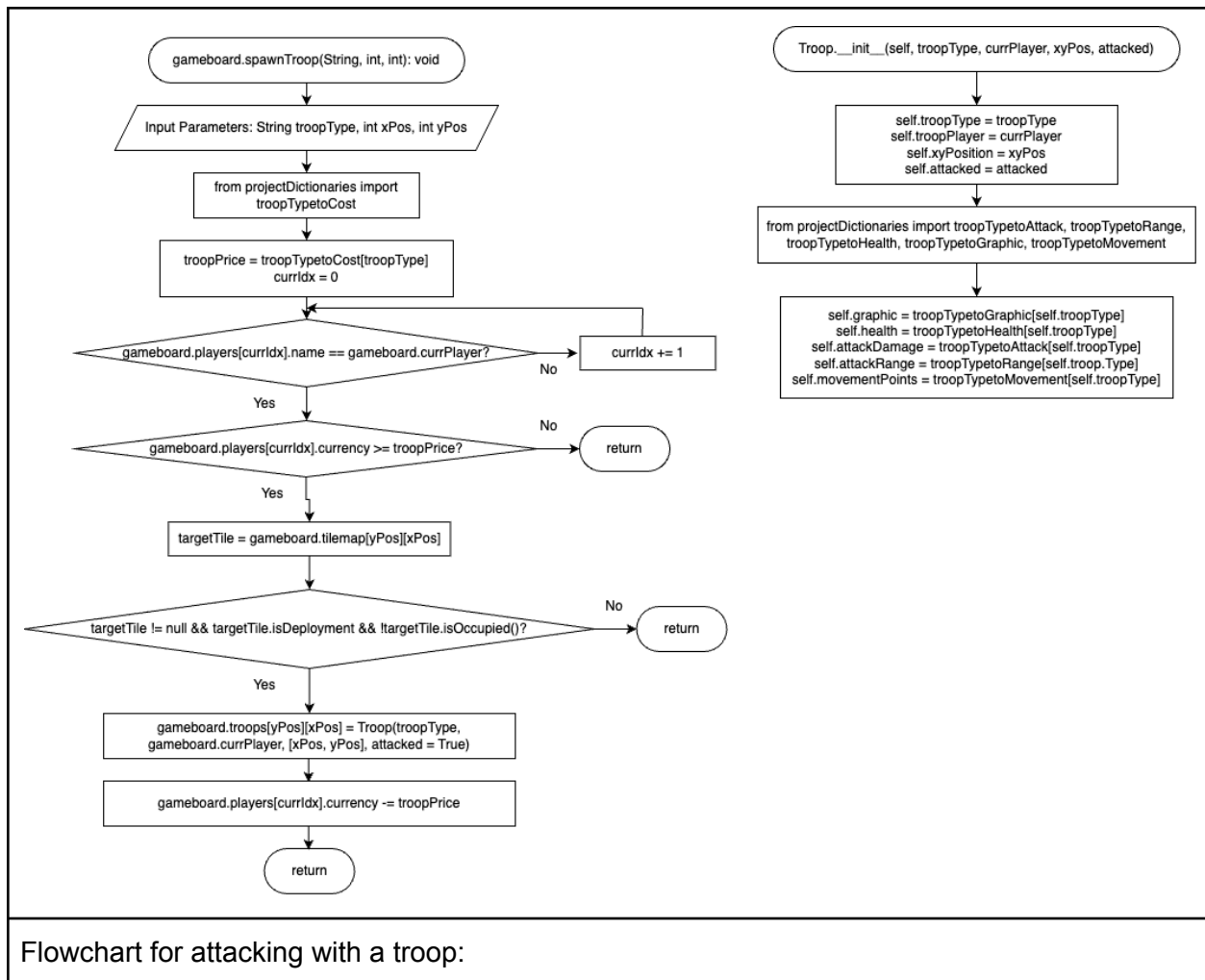
Renders button graphic, then draws rectangular outline of border width 2

UML Class Diagrams / Entity-relationship Diagram

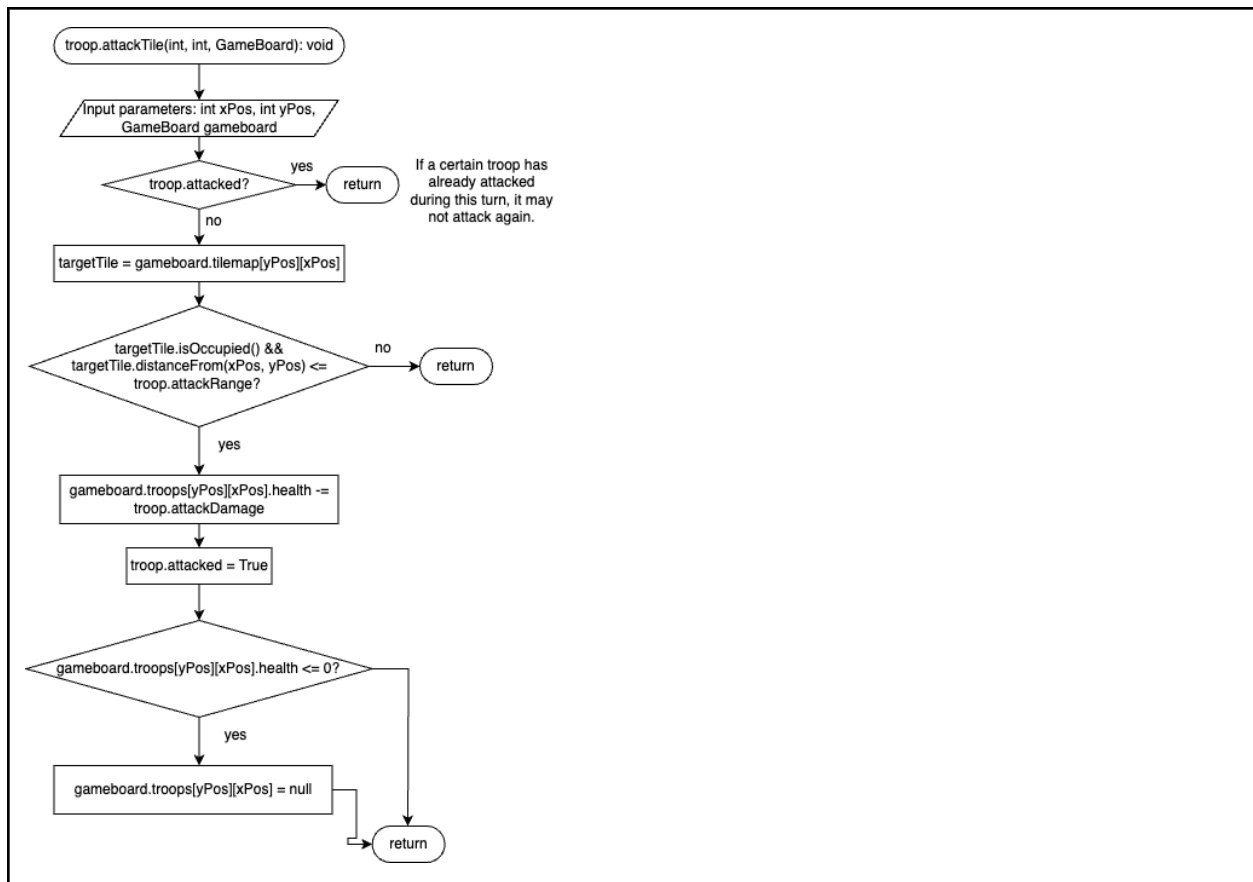


Flowchart for key methods/processes

Flowchart for spawning a troop:

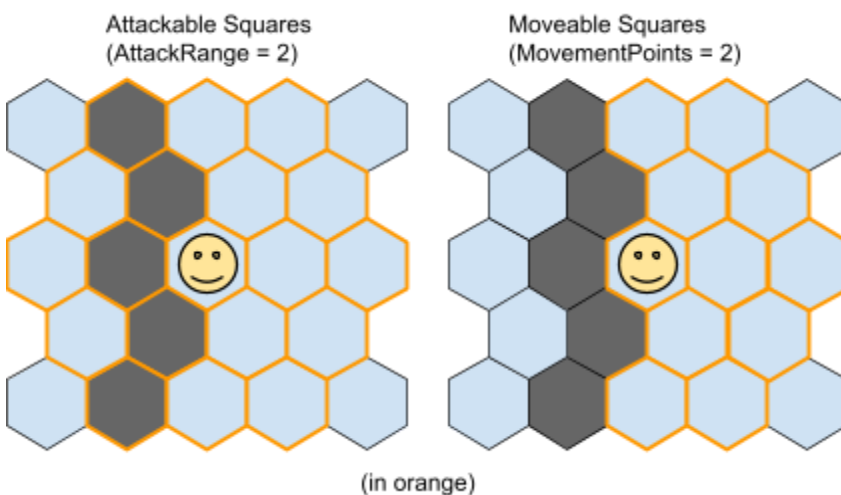


Flowchart for attacking with a troop:

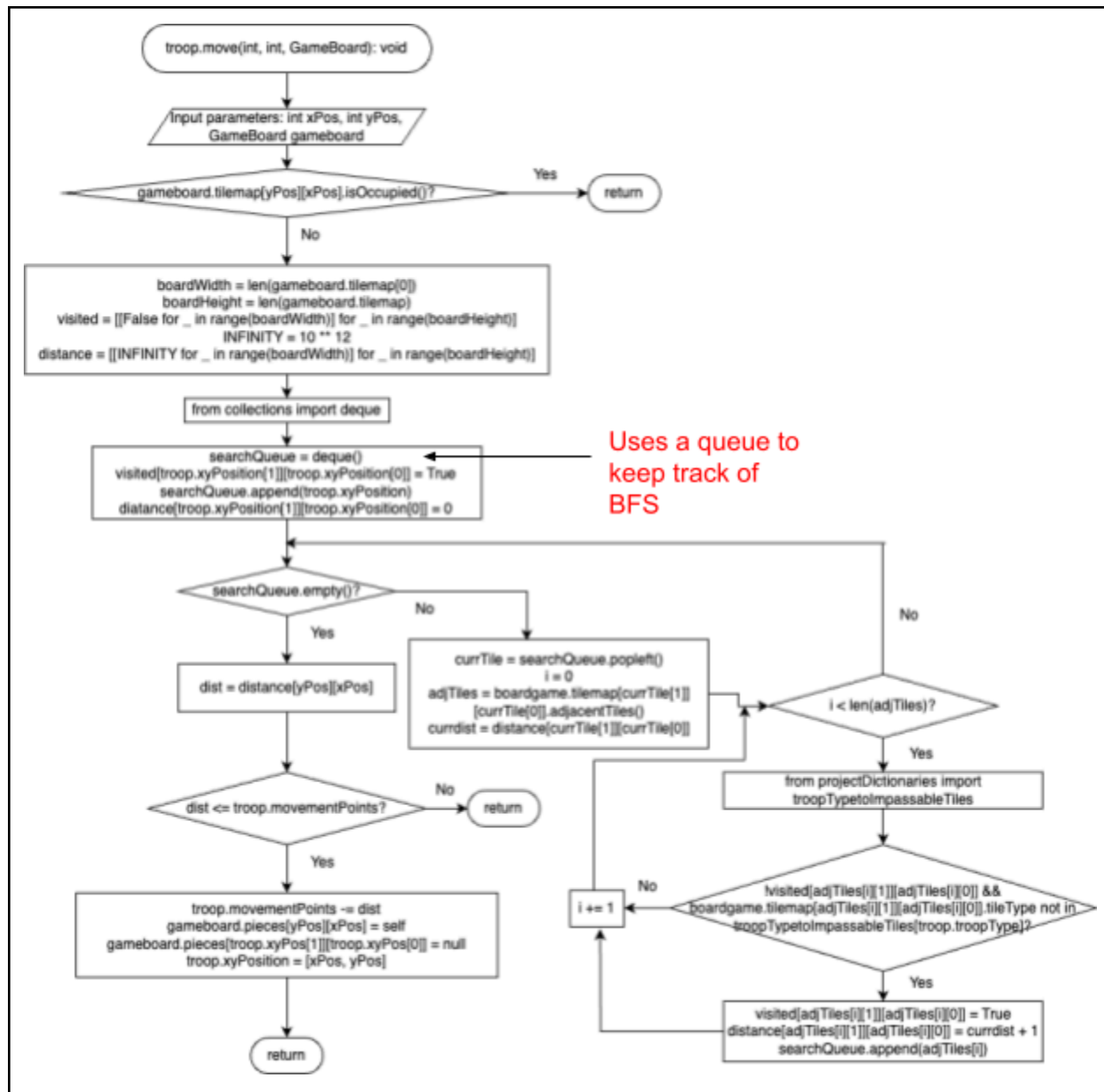


Flowchart for moving with a troop:

The difference between moving and attacking is that attacking may ignore tile-based barriers.



Therefore, `Tile.distanceFrom(int, int)`: int is insufficient for determining whether a tile can be reached. Treating the tile map as a graph, BFS can be used to determine the minimum number of adjacent steps to reach a certain tile.



Data Storage Design (map data + game state files).

Map Data:

There are 3 maps for map data, each with its tile layout stored as a csv file.
Each tile type can be represented with a character.

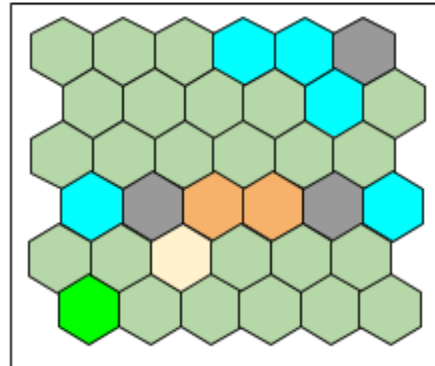
csv file

```
f, f, f, w, w, u,
f, f, f, f, w, f,
f, f, f, f, f, f,
w, u, r, r, u, w,
f, f, d, f, f, f,
p, f, f, f, f, f,
```

Initialization
of
GameBoard()
class



board appearance



The hash table/python dictionary below is used for conversion

Character encoding	Tile type
w	water
f	forest
p	plains
m	mountain
u	urban
r	rail
c	road
R	Reinforcement point

Saving Game Data:

To use JSON files to store class objects and lists of objects as data, a serialization and deserialization method must be written for every class that must be saved. Serialization and deserialization will be done by converting or deconverting from Python dictionaries/hash maps.

Pseudocode:

```

class GameBoard:
    def __init__(self, tilemap, troops, players, currPlayer):
        self.tilemap = tilemap
        self.troops = troops
        self.players = players
        self.currPlayer = currPlayer
4 usages (3 dynamic)
    def to_dict(self):
        return {
            'tilemap': [[item.to_dict() if item is not None else None for item in row] for row in self.tilemap],
            'troops': [[item.to_dict() if item is not None else None for item in row] for row in self.troops],
            'players': [item.to_dict() if item is not None else None for item in self.players],
            'currPlayer': self.currPlayer
        }
1 usage
    @classmethod
    def from_dict(cls, dict_data):
        tilemap = [[Tile.from_dict(item) if item is not None else None for item in row] for row in dict_data['tilemap']]
        troops = [[Troop.from_dict(item) if item is not None else None for item in row] for row in dict_data['troops']]
        players = [Player.from_dict(item) if item is not None else None for item in dict_data['players']]
        currPlayer = dict_data['currPlayer']
        return cls(tilemap, troops, players, currPlayer)

```

Converts each item into their serialized versions, and stores the results in a 2D or 1D list as needed.

Runs the inverse method on each item found in the dictionary data, keeping them in 2D or 1D list format.

Tile Class Example

```

class Tile:
    def __init__(self, tileType, xyPosition, isCity, isDeployment):
        self.tileType = tileType
        self.graphic = pygame.image.load(tileTypeToGraphic[tileType])
        self.xyPosition = xyPosition
        self.isCity = isCity
        self.isDeployment = isDeployment
3 usages (3 dynamic)
    def to_dict(self):
        return {
            'tileType': self.tileType,
            'xyPosition': [item for item in self.xyPosition],
            'isCity': self.isCity,
            'isDeployment': self.isDeployment,
        }
1 usage
    @classmethod
    def from_dict(cls, dict_data):
        return cls(dict_data['tileType'], [item for item in dict_data['xyPosition']], dict_data['isCity'], dict_data['isDeployment'])

```

For simpler attributes, simply store directly.

To recall the item, simply re-initialize the class with the data in the json file.

Saving and loading json file

```

gameboard = GameBoard( tilemap: [[[Tile( tileType: "plains", xyPosition: [0,0], isCity: False, isDeployment: False), None]],

# saving file
json_str = json.dumps(gameboard.to_dict(), indent=_3)

with open('data.json', 'w') as file:
    file.write(json_str)

# loading file
with open('data.json', 'r') as file:
    json_str = file.read()

dict_data = json.loads(json_str)
loaded_gameboard = GameBoard.from_dict(dict_data)

```

json module is used to generate a string, converting the gameboard information in a dictionary to a json format

Writing to files and reading to files

Json save file example

```

1  {
2  "tilemap": [
3  [
4  {
5      "tileType": "plains",
6      "xyPosition": [
7          0,
8          0
9      ],
10     "isCity": false,
11     "isDeployment": false
12     },
13     null
14     ],
15     [
16     null,
17     {
18         "tileType": "urban",
19         "xyPosition": [

```

2D list containing tile objects

Row 1

How each tile object is represented

Row 2

Test Plan for Success Criteria

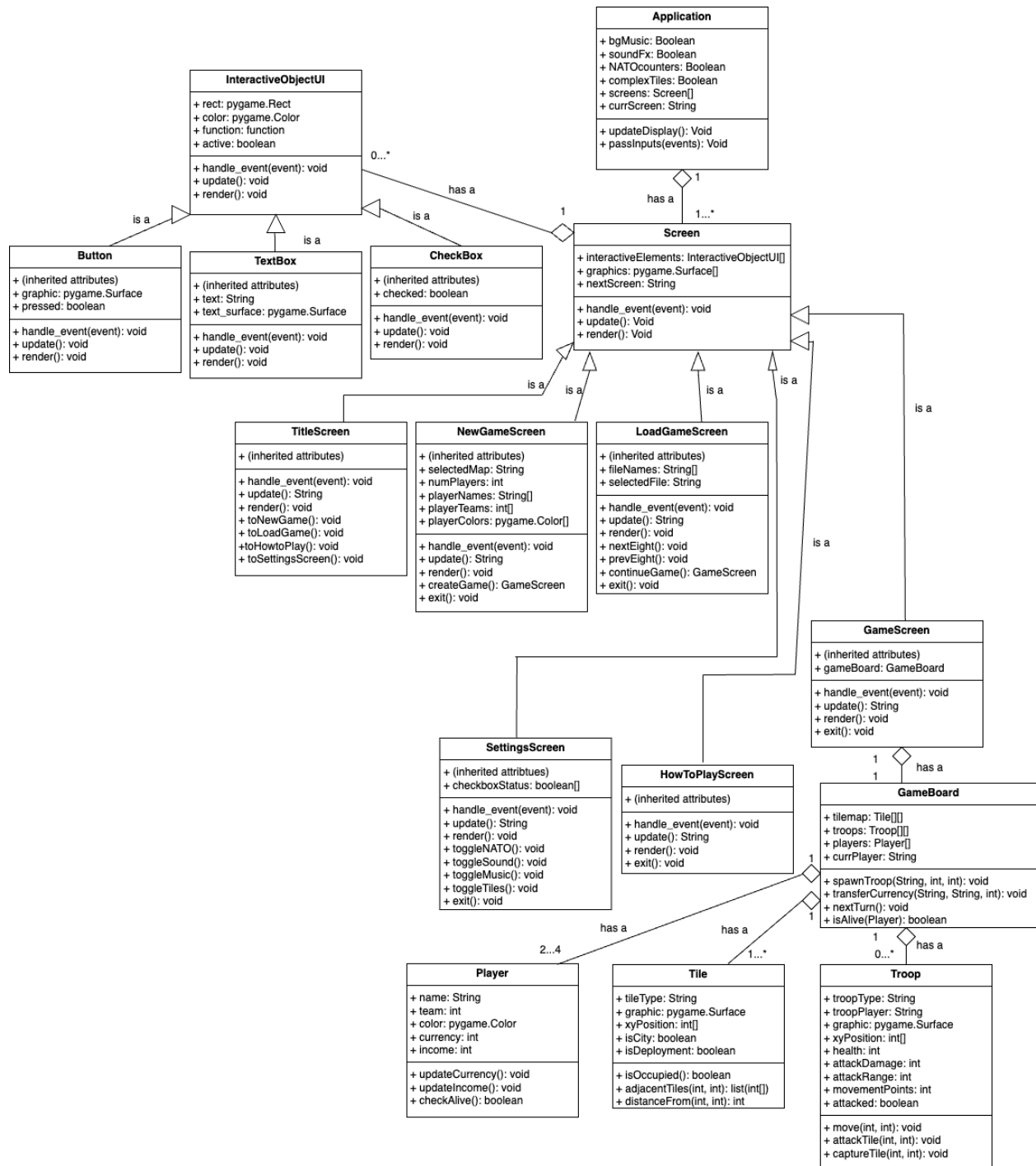
Success Criteria	Testing	Expectation
1	Open/attempt to run the program on a macOS device.	Application should successfully open, and run all functionalities as expected.
2	Go to the GameScreen, and attempt to deploy, move, and attack a tile using a troop in consecutive turns.	All of these commands should be completed successfully with clear visual indication within the game's rules.

3	During the game, save and exit. Then attempt to reopen and continue the game by navigating to the load game screen. Also attempt to save and exit, then close and reopen the program, then attempt to load the game.	The loaded game should be in the exact same state as the game when it was saved, in both cases.
4	Open the title screen.	There should be a background image loaded with a connection to a military theme.
5	Send the compressed zip file to another macOS device. Decompress and attempt to run the main program.	The application should run successfully on this device without the need for any further installation.
6	Toggle each switch in the settings page, and select each of the 3 maps in the new game screen before creating a game.	When each game is created, the effect of each setting (toggling between simple tiles and complex tiles for example) should be visually present. Furthermore, each map should be distinct in size when the game begins.
7	Select each of the 3 maps in the new game screen and create a new game.	Each map should have at least one of: field tile, urban tile, mountain tile, water tile, reinforcement points, roads, railways.
8	Create a new game, and deploy each of the troop types, and check their properties/statistics in the selected window (on the right).	Each distinct troop types should have different displayed properties, including different movement speed, health points, and attack points.
9	Perform each basic action with each troop type	A sound effect should be heard after each action. Sound effects may be the same for some actions.
10	Toggle Background music on the settings screen to 'on' and traverse between each screen.	The music should be playing continuously on all of the screens.
11	Open the application. From the title screen, test the new game, and load the game and settings screens.	The title screen should always be the first screen seen.
12	On the new game screen, set the number of players to 2, and set the players to different teams. Create game. Repeat for 3 players.	For both times the game should successfully be initialized with the correct number of players and teams.

Criterion C

Product development

UML Diagram

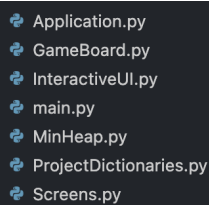


List of Techniques Used:

- A. Main Loop
- B. Class Inheritance/overwriting functions
- C. Class Aggregation
- D. Parameter Passing
- E. File loading/organization
- F. Dictionaries
- G. 2D array for tiles
- H. Greedy Distance Algorithm
- I. Modified Dijkstra's Algorithm
- J. Minimum Binary Heap

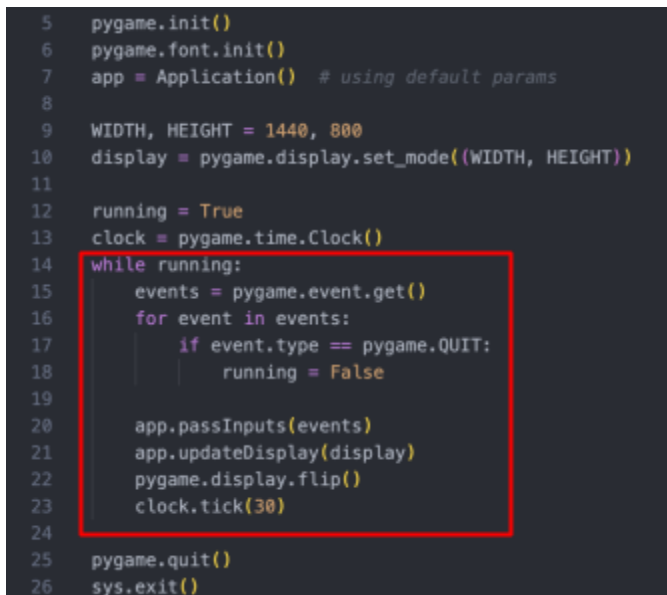
A. Main Loop

The product contains 7 Python files:



- Application.py
- GameBoard.py
- InteractiveUI.py
- main.py
- MinHeap.py
- ProjectDictionaries.py
- Screens.py

but only 'main.py' needs to be run. Below is the main loop:



```
5  pygame.init()
6  pygame.font.init()
7  app = Application() # using default params
8
9  WIDTH, HEIGHT = 1440, 800
10 display = pygame.display.set_mode((WIDTH, HEIGHT))
11
12 running = True
13 clock = pygame.time.Clock()
14 while running:
15     events = pygame.event.get()
16     for event in events:
17         if event.type == pygame.QUIT:
18             running = False
19
20     app.passInputs(events)
21     app.updateDisplay(display)
22     pygame.display.flip()
23     clock.tick(30)
24
25 pygame.quit()
26 sys.exit()
```

The Application() object is imported from 'Application.py'. The main loop is a 'while loop' contingent on the boolean 'running' being True. Every loop, it fetches events from pygame. If one event is pygame.QUIT, then running is set to false. Otherwise, events are passed to the app as a parameter, the display updates, and the loop repeats.

B. Class Inheritance/overwriting methods

For the screens in the app, the class 'Screen' serves as a template providing most functions:

```
7 class Screen:
8     def __init__(self, baseDesign):
9         self.baseDir = "media/screenDesigns/"
10        self.interactiveElements = []
11        self.nextScreen = "name"
12        self.graphic = pygame.transform.scale(pygame.image.load(self.baseDir + baseDesign), (1440, 800))
13        self.name = 'name'
14
15    def handle_event(self, events):
16        for element in self.interactiveElements:
17            element.handle_event(events)
18
19    def update(self):
20        for element in self.interactiveElements:
21            element.update()
22
23    def render(self, display):
24        display.blit(self.graphic, (0, 0))
25        for element in self.interactiveElements:
26            element.render(display)
27
28    def goToTitle(self):
29        self.nextScreen = "title"
30
31    def goToNewgame(self):
32        self.nextScreen = "newgame"
33
34    def goToLoadgame(self):
35        self.nextScreen = "loadgame"
36
37    def goToHowtoPlay(self):
38        self.nextScreen = "howtoplay"
```

The key methods are `handle_event()`, how it handles user input, `update()`, how it updates its elements, and `render()`, how it renders itself on the display.

TitleScreen inherits from Screen:

```
50 class TitleScreen(Screen):
51     def __init__(self, baseDesign="title.png"):
52         super().__init__(baseDesign)
53         self.baseDesign = baseDesign
54
55         newgameA, newgameB = generateHighlight('media/graphics/buttons/title_newgame.png')
56         loadgameA, loadgameB = generateHighlight('media/graphics/buttons/title_loadgame.png')
57         howtoplayA, howtoplayB = generateHighlight('media/graphics/buttons/title_howtoplay.png')
58         settingsA, settingsB = generateHighlight('media/graphics/buttons/title_settings.png')
59
60         self.interactiveElements = [Button(self.goToNewgame, 1032, 362, 325, 92,
61                                           [newgameA, newgameB]),
62                                     Button(self.goToLoadgame, 1032, 470, 325, 92,
63                                           [loadgameA, loadgameB]),
64                                     Button(self.goToHowtoPlay, 1032, 576, 325, 92,
65                                           [howtoplayA, howtoplayB]),
66                                     Button(self.goToSettings, 1032, 682, 325, 92,
67                                           [settingsA, settingsB]),
68                                     ]
69
70         self.nextScreen = "title"
71         self.name = "title"
```

It calls the parent class' initialization method, then instantiates the 4 interactive elements that are unique to itself. Some screens have elements with unique properties, such as NewGameScreen, which has input boxes that can appear or disappear depending on `self.numplayers`. These inherit from Screen, but overwrite certain functions:

```

142
143     def update(self):
144         for element in self.interactiveElements:
145             element.update()
146         for i in range(self.numplayers):
147             self.textBoxes[i].update()
148             self.playerTeams[i].update()
149             self.playerColors[i].update()
150

```

By using a for loop, if there are 4 textboxes, but self.numplayers = 2, then only the first 2 textboxes will be updated.

C. Class Aggregation

Most common class/object relationship is aggregation. The Application class 'has' many Screen objects:

```

5     class Application:
6         def __init__(self, bgMusic=True, soundFx=True, natoCounters=True,
7             complexTiles=True, screens=None, currScreen="title"):
8             self.bgMusic = bgMusic
9             self.soundFx = soundFx
10            self.natoCounters = natoCounters
11            self.complexTiles = complexTiles
12            self.currScreen = currScreen
13
14            if (screens is None):
15                self.screens = {"title": TitleScreen(),
16                    "loadgame": LoadGameScreen(),
17                    "settings": SettingsScreen(self.toggleCounter,
18                        self.toggleSound,
19                        self.toggleMusic,
20                        self.toggleTiles,
21                    ),
22                    "howtoplay": HowToPlayScreen(),
23                    "game": GameScreen(app=self),
24                    "gamehelp": HowToPlayScreen(ingame=True),
25                } # Dictionary to store future screen initializations
26                self.screens["newgame"] = NewGameScreen(self.screens["game"])
27            else:
28                self.screens = screens

```

Each screen 'has' interactive objects such as buttons and checkboxes, stored in a list, such as in SettingsScreen:

```

class SettingsScreen(Screen):
    def __init__(self, method1, method2, method3, method4, baseDesign="settings.png"):
        super().__init__(baseDesign)
        self.baseDesign = baseDesign

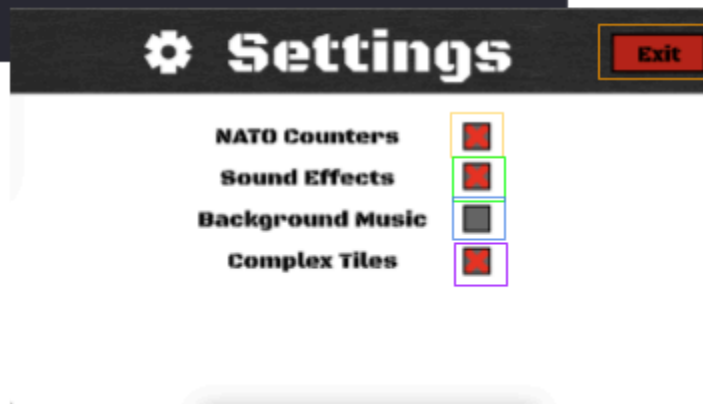
        exitA, exitB = generateHighlight('media/graphics/buttons/exit.png')
        checkA_1, checkB_1 = generateHighlight('media/graphics/checkboxes/checked.png')
        checkA_2, checkB_2 = generateHighlight('media/graphics/checkboxes/unchecked.png')

        self.interactiveElements = [
            Button(self.goToTitle, 1205, 50, 190, 80, [exitA, exitB]),
            CheckBox(method1, 908, 230, 55, 55,
                    [checkA_1, checkA_2, checkB_1, checkB_2]),
            CheckBox(method2, 908, 310, 55, 55,
                    [checkA_1, checkA_2, checkB_1, checkB_2]),
            CheckBox(method3, 908, 395, 55, 55,
                    [checkA_1, checkA_2, checkB_1, checkB_2]),
            CheckBox(method4, 908, 480, 55, 55,
                    [checkA_1, checkA_2, checkB_1, checkB_2]),
        ]

        self.nextScreen = "settings"
        self.name = "settings"

```

interactiveElements and corresponding GUI elements.



When the screen updates, it iterates through this list and updates each element.

D. Parameter Passing

After the events are passed by the main loop to the Application class, it passes it as a parameter to the current screen:

```

39     def passInputs(self, events):
40         self.screens[self.currScreen].handle_event(events)

```

The Screen class passes on the events to each of its interactive elements:

```

15     def handle_event(self, events):
16         for element in self.interactiveElements:
17             element.handle_event(events)

```

Finally, the interactive element processes, below is the method used by the Button class:

```

def handle_event(self, events):
    for event in events:
        if event.type == pygame.MOUSEBUTTONDOWN:
            if self.rect.collidepoint(event.pos):
                self.pressed = True
        if event.type == pygame.MOUSEBUTTONUP:
            if self.pressed:
                self.pressed = False
                self.active = True

        if (self.rect.collidepoint(pygame.mouse.get_pos())):
            self.hover = True
        else:
            self.hover = False

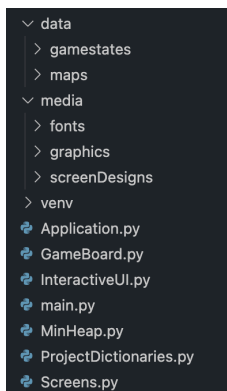
```

The Button class overwrites the `handle_event` method inherited from `InteractiveObjectUI`. The button's pressed state is true when the mouse button is down, but is inactive until the mouse button is released.

In Python, as 'events' is a complex object, only its reference is being passed as a parameter.

E. File loading/organization

Below is the file organization of the project. Media stores graphics and fonts. Data stores saved games, and map layouts.



To read the json files within the 'data/gamestates' folder, the `LoadGameScreen` lists the files in that directory, sorts the files, and creates an interactive `SelectBox` element for each:

```

self.files = os.listdir("data/gamestates")
self.files.sort()
self.fileCount = len(self.files)

self.startidx = 0

self.selectBoxes = []

x = 266
y = 284
h = 56
w = 910

for i in range(self.startidx, self.startidx + min(self.fileCount, 8)):
    self.selectBoxes.append(SelectBox(x, y + i * h, w, h, [fileboxA, fileboxB],
                                     self.files[i], pygame.Rect(x, y, w, h * 8)))

```

When loading the map data in csv format, each character is logged into a 2D list using the following loop:

```

self.tiletypes = [] # stores the tile type for each row and column
self.tilesizes = maptypetoTileSize[mapfile]

with open(self.filepath, newline='') as csvfile:
    csvreader = csv.reader(csvfile)

    # Loop through each row in the CSV file
    for row in csvreader:
        # Append the row to the data list
        self.tiletypes.append(row)

# Now 'data' is a 2D list representing the CSV file
# print(self.tiletypes)

```

The csv reader (which is imported from a module), allows the data in each file to be converted efficiently, as it can be iterated through by row.

The Pygame module provides tools for loading, augmenting, and displaying graphics. I made the following function to load unhighlighted and highlighted versions of a graphic, useful for GUI elements:

```

54 def generateHighlight(filepath, highlight = 50):
55     imageA = pygame.image.load(filepath)
56     imageB = pygame.image.load(filepath)
57     imageB.fill((highlight, highlight, highlight), special_flags=pygame.BLEND_RGB_ADD)
58     return imageA, imageB

```

F. Dictionaries

Python dictionary is a data structure that allows data to be accessed with a string. I used it to hold easily customizable properties such as the RGB colors of tiles:

```

17  ▾ tiletypetoColor = {
18      'f': (0, 102, 0),
19      'w': (0, 255, 255),
20      'p': (102, 255, 51),
21      'm': (102, 153, 153),
22      'u': (51, 102, 153),
23      'r': (204, 102, 153),
24      'c': (153, 51, 51),
25      'R': (255, 153, 102),
26  }

```

Given a tile character, it returns a tuple of values, representing the color of said tile.

I also nested dictionaries:

```

36  move_cost = {
37      "infantry": {'f': 1, 'w': 99, 'p': 1, 'm': 1, 'u': 1, 'r': 1, 'c': 0.5, 'R': 1},
38      "mechanized": {'f': 1.5, 'w': 99, 'p': 1, 'm': 99, 'u': 1, 'r': 1, 'c': 0.5, 'R': 1},
39      "helicopter": {'f': 1, 'w': 1, 'p': 1, 'm': 99, 'u': 1, 'r': 1, 'c': 1, 'R': 1},
40      "tank": {'f': 1.5, 'w': 99, 'p': 1, 'm': 99, 'u': 1, 'r': 1, 'c': 0.5, 'R': 1},
41      "artillery": {'f': 1.5, 'w': 99, 'p': 1, 'm': 99, 'u': 1, 'r': 1, 'c': 0.5, 'R': 1},
42  }

```

This dictionary contains values, which themselves are dictionaries. This functions like a string-indexed table which returns the cost of movement, given a troop type and a tile type.

G. 2D arrays for tiles

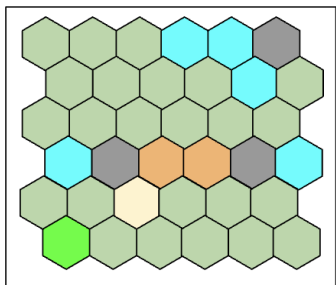
Tile objects are stored in a 2D list, indexed [row][col], and created using the following nested for loop:

```

self.tiles = []
for i in range(len(self.tiletypes)):
    row = []
    for j in range(len(self.tiletypes[i])):
        row.append(Tile(self.tiletypes[i][j], self.xyPos, self.tilesizesize, (i, j)))
    self.tiles.append(row)

```

The gameboard is hexagonal, and tessellation is achieved by offsetting even-indexed rows to the left in the formation below:



The GameBoard class has auxiliary methods for the hexagonal geometry of the board, such as `self.neighbours()` which returns all neighbour tiles given a (row, col) index:

```

def neighbours(self, index):
    # return a list of all the neighbors of an index (row, col).
    if index[0] % 2 == 1: # odd index row
        neighbours = [(index[0]-1, index[1]),
                      (index[0]-1, index[1]+1),
                      (index[0], index[1]+1),
                      (index[0], index[1]-1),
                      (index[0]+1, index[1]),
                      (index[0]+1, index[1]+1),
                      ]
    else:
        neighbours = [(index[0]-1, index[1]-1),
                      (index[0]-1, index[1]),
                      (index[0], index[1]+1),
                      (index[0], index[1]-1),
                      (index[0]+1, index[1]-1),
                      (index[0]+1, index[1]),
                      ]
    bound_neighbours = []
    # basically, remove any that are out of bounds, before returning the value.
    for pair in neighbours:
        if pair[0] < 0 or pair[1] < 0:
            continue
        if pair[0] >= len(self.tiles) or pair[1] >= len(self.tiles[0]):
            continue

        bound_neighbours.append(pair)
    return bound_neighbours

```

It identifies the index of neighbours depending on the parity of the input row, before removing any indices that are outside the bounds of the gameboard.

When a method needs to process an interaction involving tiles, the (row, col) indices are passed as parameters in methods:

```

def tile_interaction(self, actorindex, reactorindex):
    # this function will handle any tile acting on any other tile.
    print(f"{self.players[self.turn].name}: {actorindex} ACTS UPON {reactorindex}")

    #for now, as a place holder, can make a very simple move command here.
    if self.tiles[reactorindex[0]][reactorindex[1]].troop is None:
        self.move_troop(actorindex, reactorindex)
    elif self.tiles[reactorindex[0]][reactorindex[1]].troop.ownerid != self.turn:
        self.atk_troop(actorindex, reactorindex)

```

If the tile at reactorindex has no troop, then a move command is issued. If there is a troop not belonging to the current player, then an attack command is issued.

H. Greedy Distance Algorithm

In a square tiling, the 'manhattan distance' between 2 tiles could be calculated in constant time as $\text{abs}(y_2 - y_1) + \text{abs}(x_2 - x_1)$. As the board is hexagonal, I created a linear time algorithm to traverse greedily from one tile to the other:


```

def tile_distance(self, index_a, index_b):
    index_a = list(index_a)
    index_b = list(index_b)
    # given 2 sets of (row, col) indices, find the min distance between.
    oddrow = index_a[0] % 2
    dist = 0
    # use a greedy algorithm to most efficiently traverse from a to b.
    #Note that the valid moves on an odd row are: (-1,0), (-1,1), (0,1), (0,-1), (1,1), (1,0)
    #and on an even row are: (-1,0), (-1,-1), (0,1), (0,-1), (1,-1), (1,0)

    while index_a != index_b:
        if oddrow:
            #if a and b are alr on the same row, no need can just move in approp. dir.
            if (index_a[0] == index_b[0]):
                if (index_a[1] < index_b[1]):
                    index_a[1] += 1
                else:
                    index_a[1] -= 1

            elif (index_a[0] < index_b[0]):
                if (index_a[1] < index_b[1]):
                    index_a[1] += 1
                    index_a[0] += 1
                else:
                    index_a[0] += 1

            else: # index_a[0] > index_b[0]:
                if (index_a[1] < index_b[1]):
                    index_a[1] += 1
                    index_a[0] -= 1
                else:
                    index_a[0] -= 1

            oddrow = index_a[0] % 2
        else:
            if (index_a[0] == index_b[0]):
                if (index_a[1] < index_b[1]):
                    index_a[1] += 1
                else:
                    index_a[1] -= 1

            elif (index_a[0] < index_b[0]):
                if (index_a[1] > index_b[1]):
                    index_a[1] -= 1
                    index_a[0] += 1
                else:
                    index_a[0] += 1

            else: # index_a[0] > index_b[0]:
                if (index_a[1] > index_b[1]):
                    index_a[1] -= 1
                    index_a[0] -= 1
                else:
                    index_a[0] -= 1

            oddrow = index_a[0] % 2
            dist += 1
    return dist

```

Each iteration, the algorithm selects the neighbouring position that brings index_a the closest to index_b at that step. dist increments by 1, and the loop repeats until index_a == index_b. This method is used when determining whether a troop's attack target is within range:

```
def atk_troop(self, actorindex, reactorindex):

    if self.tiles[actorindex[0]][actorindex[1]].troop and not self.tiles[actorindex[0]][actorindex[1]].troop.baby and self.tiles[actorindex[0]][actorindex[1]].troop.range and self.tiles[actorindex[0]][actorindex[1]].troop.health >= self.tiles[actorindex[0]][actorindex[1]].troop.atk_pts:
        self.tiles[actorindex[0]][actorindex[1]].troop.health -= self.tiles[actorindex[0]][actorindex[1]].troop.atk_pts
        self.tiles[actorindex[0]][actorindex[1]].troop.atk_pts -= 1
        # remove troop from board if health is less than 0 after attack.
        if self.tiles[actorindex[0]][actorindex[1]].troop.health <= 0:
            self.tiles[actorindex[0]][actorindex[1]].troop = None
```

I. Modified Dijkstra's Algorithm

Dijkstra's algorithm finds the lowest cost to reach every node of a positively weighted graph from some starting node, and when implemented with a priority queue, runs in $O((V+E)\log V)$. Below is my implementation based on code online (Geeks for Geeks):

```
def dijkstra_distance(self, index_a, trooptype):
    # use dijkstra's algorithm to find the shortest distance of every tile from index_a given just the trooptype.
    # return a 2D list object
    # https://www.geeksforgeeks.org/dijkstras-shortest-path-algorithm-greedy-algo-7/

    distances = [[sys.maxsize for i in range(len(self.tiles[0])) for j in range(len(self.tiles))]]
    distances[index_a[0]][index_a[1]] = 0

    # just guesstimated that 10,000 is a big enough heap for the tiles in this game, could expand if really big map (over 100 by 100)
    pq = MinHeap(10000)
    pq.insert([0, (index_a[0], index_a[1])])
    # print(pq.size)
    while pq.size > 0:
        dist, index = pq.extract_min()
        # find all the neighbors of index.
        # print(self.neighbours(index))
        for i in self.neighbours(index):
            # basically, ignore as a neighbour if there is a troop (of any player) chilling there, unless helicopter which can fly
            # ok, so the gist is that helicopters cannot block units.
            if trooptype != "helicopter" and self.tiles[i[0]][i[1]].troop and self.tiles[i[0]][i[1]].troop.type != "helicopter":
                continue

            if distances[i[0]][i[1]] > distances[index[0]][index[1]] + move_cost[trooptype][self.tiles[i[0]][i[1]].tileType]:
                distances[i[0]][i[1]] = distances[index[0]][index[1]] + move_cost[trooptype][self.tiles[i[0]][i[1]].tileType]
                pq.insert([distances[i[0]][i[1]], i])

    return distances
```

One key modification is that my tiles are indexed using (row, col), so the distance values were stored on a 2D list indexed (row, col). I specifically ignored tiles obstructed with ground troops, in red. Instead of explicitly storing a graph, I used `self.neighbours()` to identify edges, and fetched values of weights from the nested dictionary in section F. This allows for dynamic calculation of which tiles a troop can reach in its turn.

J. Minimum Binary Heap

Without an efficient priority queue, Dijkstra's runs in $O(V^2)$, which is inefficient for large values of V . I implemented a binary heap data structure to serve as an efficient priority queue, based on code from online (Geeks for Geeks).

```

2 # https://www.geeksforgeeks.org/binary-heap/ used as reference for methods needed
3 class MinHeap:
4     # Using a binary heap.
5     def __init__(self, size):
6         # this is my array for the stuff, for now will just give it a static size.
7         # assume each item is a (key, value) pair, we are sorting by key.
8         self.array = [None for i in range(size)]
9         self.size = 0
10
11     def insert(self, x):
12         self.array[self.size] = x
13         self.size += 1
14         self.bubble_up(self.size - 1)
15
16     def peek(self):
17         if self.size == 0:
18             return None
19         return self.array[0]
20
21     def extract_min(self):
22         if self.size == 0:
23             return None
24         minval = self.array[0]
25         self.array[0] = None
26         self.swap(0, self.size - 1)
27         self.size -= 1
28         self.bubble_down(0)
29         return minval
30
31     def swap(self, idx1, idx2):
32         temp = self.array[idx1]
33         self.array[idx1] = self.array[idx2]
34         self.array[idx2] = temp

```

```

36     def bubble_up(self, idx):
37         # (idx - 1) // 2 is the index of the parent node.
38         parent = (idx - 1) // 2
39         while idx > 0 and self.array[idx][0] < self.array[parent][0]:
40             self.swap(idx, parent)
41             idx = parent
42             parent = (idx - 1) // 2
43
44     def bubble_down(self, idx):
45         # children nodes are at 2*idx+1, and 2*idx+2
46         while True:
47             leftidx = 2 * idx + 1
48             rightidx = 2 * idx + 2
49             smallestidx = idx
50
51             if leftidx < self.size and self.array[leftidx][0] < self.array[smallestidx][0]:
52                 smallestidx = leftidx
53             if rightidx < self.size and self.array[rightidx][0] < self.array[smallestidx][0]:
54                 smallestidx = rightidx
55
56             if smallestidx != idx:
57                 self.swap(idx, smallestidx)
58                 idx = smallestidx
59             else:
60                 break

```

I omitted unnecessary methods of the normal implementation, keeping only methods needed for dijkstra's algorithm, and modified comparators to assume each item is a list with the structure: [distance, (row,col)].

Criterion D

Video demonstrating success criteria

(5 - 7 min)

Extensibility of the product

Criterion E

Evaluation

(250 words)

Recommendations for improvement

(250 words)

Appendix

Appendix a - First and second interviews with client

[a.i.] First interview with client

(Discuss the client's needs)

(Transcript)

a.i.i

Q1: What is one way in which a piece of software, like a website or an app for instance, can potentially improve or address a specific problem that you have encountered as of late?

Client: As of late, I have encountered a problem of not having a means to play with my friends online as a videogame, specifically in regards to an original idea that I had for a turn-based strategy game that I can play with my friends. I think that an app could address that problem by creating a functional way of playing said game with my friends.

a.i.ii

Q2: What in your current situation is the greatest challenge or barrier to be able to achieve your desire of playing said game with your friends?

Client: Well the current biggest barrier is the game's existence. Other mediums that could portray the game are suboptimal in different ways. If it's coded as an app, it's easy to use, and easy to do. If it's a board game, or some other medium, then it's also not very easy to access compared to a digital product that could theoretically be downloaded from online.

a.i.iii

Q3: What are the most important aspects that a digital product can offer that's specific to your problem?

Client: Ease of access first of all. I guess, replayability. Mobility, in the sense that a game can be easily brought anywhere with a computer, whereas a proper board game would be more annoying to transport. Digital products also have saveability, in that I can easily save the state of a game when it's not finished, whereas that option doesn't exist as much for physical games.

a.i.iv

Q4: How did you generally come up with the idea for the game in the first place? Do you have any points of inspiration from other games?

Client: I guess I got inspiration from the tank game, "Tanks of Freedom", a Ukrainian indie game. I've been making games on paper as a child for a while now, when I used to force my parents to play those games with me. In terms of aesthetic visions, there are a lot of WW2 tabletop games that visually appeal to me, which I would want my game idea to represent or reflect. Another inspiration is a video game, "Glory of Generals 3", which works sort of similarly.

a.i.v

Q5: Are there any other details regarding your general needs that you think should be addressed?

Client: It's gotta be fun. It's gotta be cool.

[a.ii.Second interview with client]

(Discuss the success criteria for the final product)

a.ii.i

Q1: So how does your game work exactly? What do you have in mind in terms of the general game mechanics and systems that you would like to see digitized?

Well, I want like a stream-lined combat system, because otherwise it's a pain making moves and stuff with physical games often. Hopefully digitization of the game will make these mechanics automatic. I want game mechanics in terms of some sort of terrain feature in the game to be implemented, some examples I would like to see are urban, field, mountain, water tiles. You have to move pieces between hexagonal tiles. Another system I would like to employ is the reinforcement point system, so basically what I want is for players to feel constrained, and have to think around these constraints and play to their strengths and weaknesses due to these constraints. That's why I also have these terrain tiles. The point of the RP (reinforcement points) is that when you purchase new troops you can only deploy them at the RPs, from store to map. There will be RPs spread out at the walls or edges of the map, and it modifies whatever tile is already there so that it can now deploy troops. Money is earned by controlling urban tiles. I'm not completely sure how I'd like the money system to be implemented. I want players to compete for map control, which could be supported with an income system, so maybe I'd want a base income + income from controlling parts of the map. But I also don't want it to be too snowballish, there should be opportunities for players to make comebacks and for the game to be close all the way to the end.

Another point about the terrain is that I want movement bonuses, so urban and field tiles can have roads and rails placed on top of them to allow for faster travel. This way the game can simulate supply lines and logistics. I want the game to have a medium-ish pace.

a.ii.ii

Q2: It sounds like the game is multiplayer to some extent. Around how many players do you see playing the game at once, how do you expect them to interact with each other in-game? Would you need the game to work across multiple platforms and devices or connect online in some way?

OK first part, I'd say about 2 - 3 players is a fair amount. Realistically, I think 1v1 would act as standard gameplay, and 1v1v1 would be an alternate gamemode that would have far more factors for each player. As a turn-based strategy game it also would take far too long if there are any more players though, I played a strategy game with my friend some years back and it took like 20-30 minutes or maybe even an hour to finish even one game, so realistically we wouldn't want to push past say 4 players.

I want players to think strategically, play deception and mind games with each other, there wouldn't necessarily be completely concealed information from each other, but I think the mind games will be based more on mobility and freedom of movement rather than playing mind games based on unseen information. Ultimately, the game objective is total annihilation, destroying the enemies/other sides' troops. There should also be a surrender button allowing players to forfeit games.

Players can also potentially send each other in-game money to strategize.

Connection between players online or offline would be completely fine. I am fine with physical pass and play which would work offline as well.

a.ii.iii

Q3: What are you looking for aesthetically in the digital product? Are there any general styles, color palettes, etc. that would most complete the game's theme?

I'd say, first refer to the games I'm taking inspiration from. Next, there should be a general militaristic style. Furthermore, I want a military map style appearance to the game board. Therefore I want the pieces to look like NATO counters, which generally appear like rectangles or shapes with lines and logos to represent different troop types. I would want the game to have a sort of birds' eye view. The urban tiles should look like buildings on a tile for instance. Aesthetically and visually it should be clear what things are.

a.ii.iv

Q4: Thinking about the app or product as a whole, what sort of UI features would be important to have?

Well, I feel like I want a title screen that has a soldier, a title, a few basic buttons such as making a new game, saves, and settings. I want the UI in-game to work by clicking on the troops to select, and then click them somewhere else to perform some kind of action. I want visually to be able to see the terrain as well as the modifier of the terrain tiles. Imagine chess, but different squares have certain buffs, debuffs or barriers. Now imagine between the urban squares, if both have rails and are controlled by the player, the player can instantly move between them. Tile control isn't very important except in urban tiles and railroad tiles.

There's gonna be a menu, and if you click the menu you can go to the shop, and you can buy units basically that can be immediately deployed to an RP when it is your turn. In the interface, you're going to have to know who's turn it is, what turn it is and how much money you have (as well as income per turn).

a.ii.v

Q5: Are there any other extra effects that you need for the game? Do you need music, do you need sound effects, do you need any sorts of animations?

No animations, but I do want sound effects for when you move pieces. If you move like a tank piece, you need tank noises, if you move helicopter piece, you need helicopter noises, if you move an infantry, you need footstep noises. Music would be fun, just like royalty free music or media would be fine. When 2 units fight, maybe like generic gun noises, there are probably plenty of stock sounds that could be used there.

a.ii.vi

Q6: How do the pieces work then? What are the main piece types and how do they move and interact, for example?

A unit has some attack, some health, some movement. Tanks should have high attack, high health and medium movement, but be restricted by terrain such as mountains and water.

Motorized infantry should have low attack power, medium health and medium movement
Infantry should have low attack power, medium health and low movement

Artillery should have high attack, low health and low movement and also be restricted by terrain.

Helicopters have medium attack, medium health and high movement and are not restricted by movement.

Tanks are expensive, helicopters are even more expensive. Infantry and motor-infantry are a bit cheaper, and artillery is similar to infantry in cost. Tanks and artillery cannot hit helicopters.

All unit tiles can attack each other only from when they are on adjacent tiles, with the exception of the artillery which can attack even if there are up to 3 tiles in between the artillery and its target. The game should be dynamic, focused on rapid movements.

There should also be a mechanized Infantry (medium attack, medium-high hp, medium movement) which acts as a counterbalance to the helicopter. It is similarly restricted by terrain.

When starting a game and adjusting the settings, there should be an option to toggle whether each unit can be built.

a.ii.vii

Q7: How should maps/boards work? Should there be a selection/options, if so how should these vary?

I think there should be 3 main map options, where you can choose between 3 maps, one small, one medium and one large. A small board would be like a 1v1 focused map. With maybe around 10 tiles wide and 18 tiles high. The whole map is visible at once in the user interface.

The medium map should be designed with 1v1v1s or 1v1s in mind. The map should be built in a sort of Y like shape with a circular perimeter. Medium map may be around 23 by 23.

Large map for 1v1v1s, may have a triangular shape. May have water tile elements such as rivers and oceans. 30 by 30 roughly in terms of dimensions. Games should maybe take around 2 hours to complete on the large map perhaps.

a.ii.viii

Q8: Do you need anything like rescaleable application windows or other ease of life features?

Rescaleable application windows would be nice, maybe not completely necessary given the way the game works, but could be nice to have. Scroll bars, or some way of zooming into the map especially for the larger boards would be nice to have. Other than that, not really.

a.ii.ix

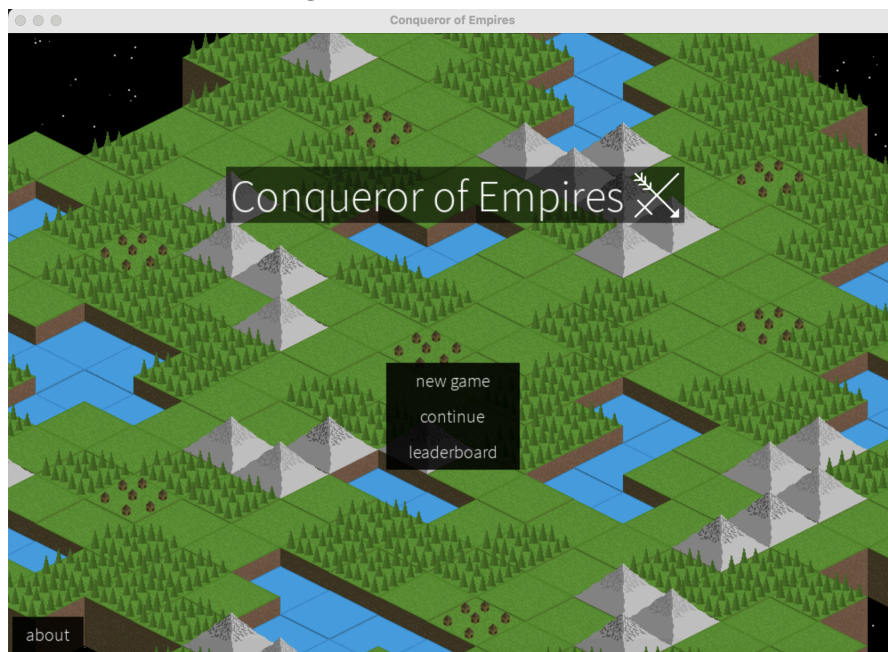
Q9: Do you need all the contents of the app to be together in one package?

It would be nice for the product to be presented as a zip file or a singular, one unit product that I can run.

Appendix b - Example product: Conqueror of Empires by Ben Ryder and Miklos Horvath

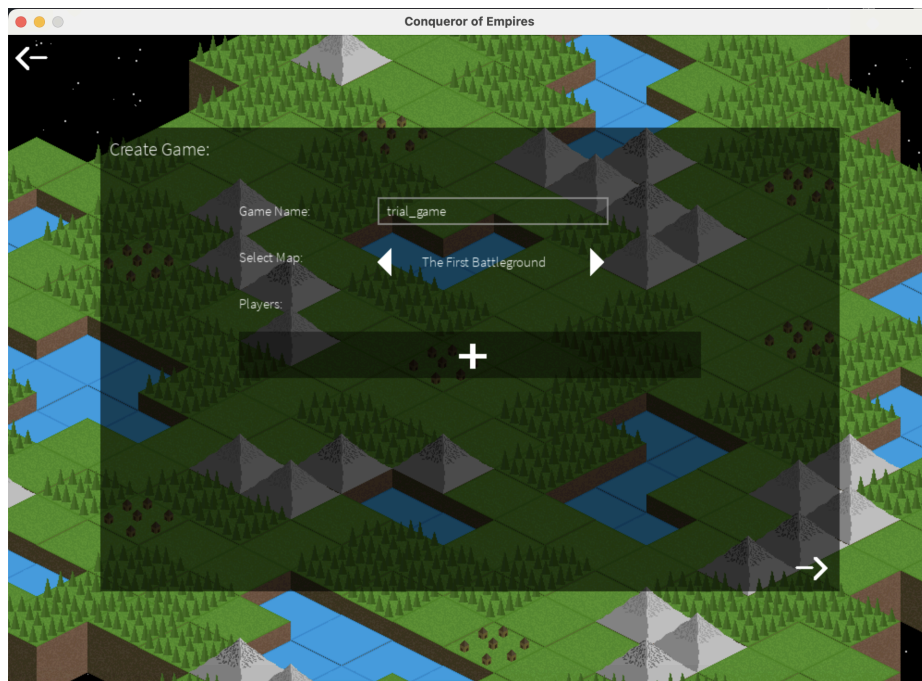
b.i.

Screenshot of title page



b.ii.

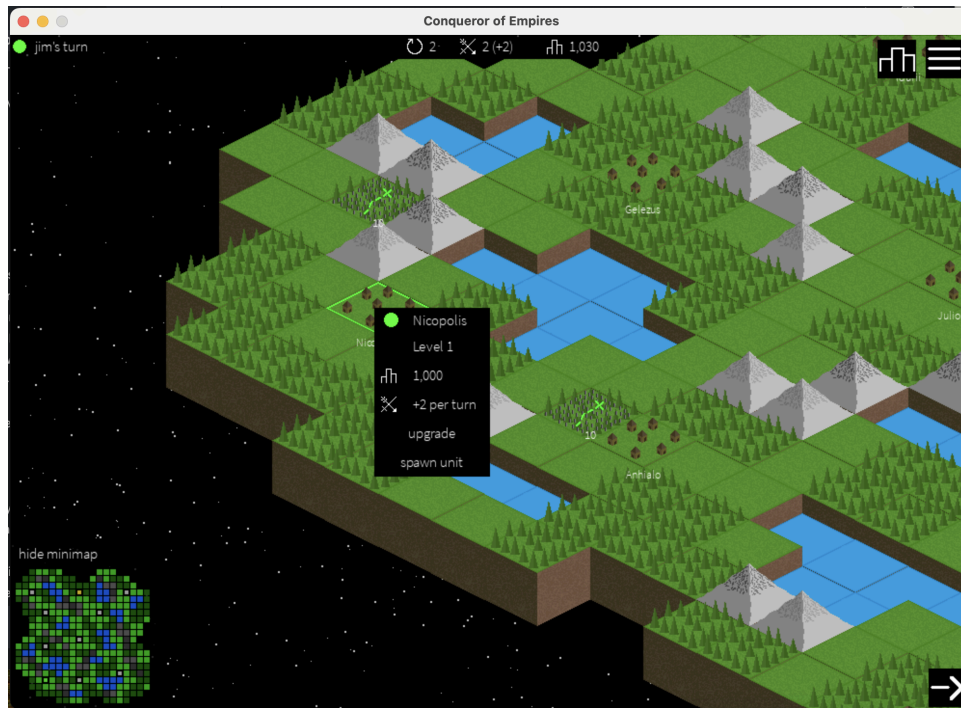
Screenshot of new game menu



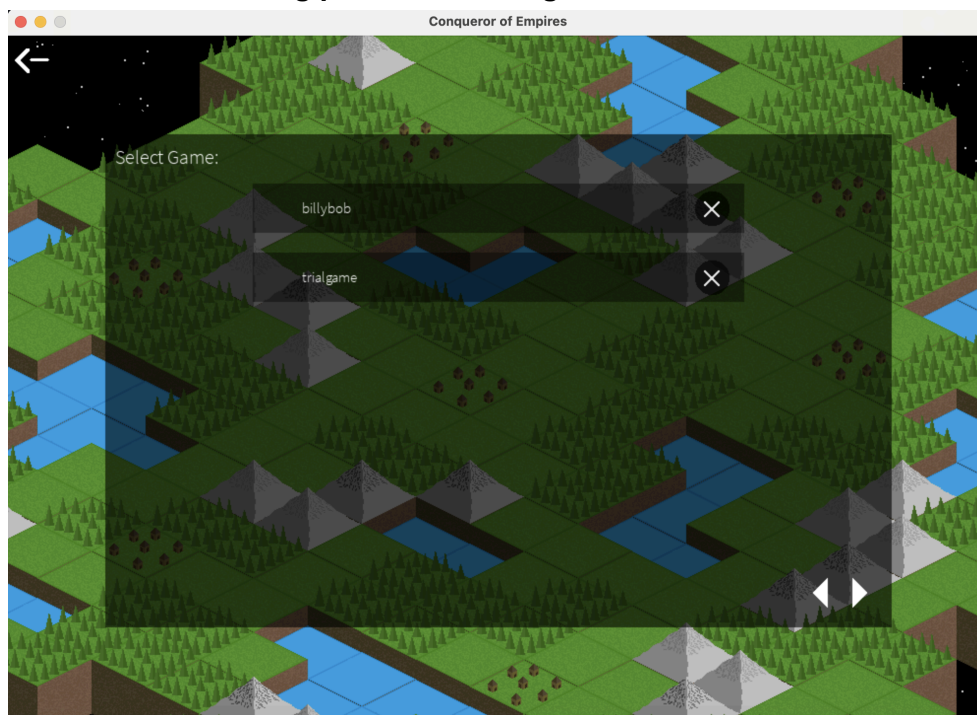
b.iii.
Screenshot of gameplay 1



b.iv.
Screenshot of gameplay 2



b.v.
Screenshot of loading previous saved games feature



Bibliography

"About - Wiki." Pygame, www.pygame.org/wiki/about. Accessed 29 Apr. 2024.

GeeksforGeeks. "Binary Heap." GeeksforGeeks, 4 Nov. 2024,
www.geeksforgeeks.org/binary-heap/.

GeeksforGeeks. "Find Shortest Paths from Source to All Vertices Using Dijkstra's Algorithm."
GeeksforGeeks, 6 Aug. 2024,
www.geeksforgeeks.org/dijkstras-shortest-path-algorithm-greedy-algo-7/.

Raval, Nihar. "Python vs JavaScript: Which Language Is Right for Your next Project?" *Radixweb*,
Radixweb, 20 Mar. 2024, radixweb.com/blog/python-vs-javascript.

Ryder, Ben, and Miklos Horvath. "Conqueror-of-Empires." GitHub, 4 Sept. 2021,
github.com/ben-ryder/Conqueror-of-Empires?tab=readme-ov-file.